UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:

Integración de estructuras de control (ciclos y controles de datos) en Java aplicada a la solución de problemas algorítmicos.

Nivel y Paralelo:	Primero “B”
Alumnos participantes:	Cabrera Ortiz Alex Sebastián Cordonez Chisag Erick Josué. Garces Salazar Kevin Javier. Gómez Toasa Cristian Alejandro Zambrano Chicaiza Carlos Esteban
Asignatura:	Algoritmos y Lógica de programación.
Docente:	Ing. Caiza Caizabuano José Rubén.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivo:

General:

Específicos:

- Analizar cada problema planteado (calificaciones, tablas de multiplicar, cajero automático, estacionamiento, cafetería, encuestas y parqueadero) identificando sus entradas, procesos y salidas, para determinar la estructura de repetición más adecuada en cada caso.
- Implementar en Java los algoritmos diseñados, aplicando validación de datos, manejo de contadores y acumuladores, y diferenciando casos normales de casos límite y datos incorrectos.
- Documentar y verificar el correcto funcionamiento de cada solución mediante pruebas de escritorio tablas de trazas, depuración de errores y el registro del proceso de desarrollo a través de commits en un repositorio de GitHub.

2.2 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐ Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐ Recursos audiovisuales
☐ Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____GITHUB_____

Enlace github:
https://github.com/EstebanZambrano2005/Ejercicios-1---10

2.3 Actividades por desarrollar

INFORME DE TRABAJO GRUPAL - APE ESTRUCTURAS DE CONTROL EN JAVA

Desarrollo Completo de los 10 Ejercicios con Metodología Obligatoria
Grupo #02

Nombre	Rol	Usuario GitHub	Tarea
Kevin Garcés	Analista	@Javi2145	Análisis del problema
Cristian Gómez	Programador	@Cristian-GT2006	Diseño del algoritmo
Erick Cordonez	Diseñador	@Erick-100	Diseño de diagrama de flujo
Esteban Zambrano	Desarrollador	@EstebanZambrano2005	Diseño de Pseudocódigo
Alex Cabrera	Desarrollador	@Alex2840	Codificación

Ejercicio 1. Control de calificaciones

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Número de estudiantes (N), calificaciones de cada estudiante.
Procesos: Validar que N > 0 y notas entre 0 y 10. Acumular calificaciones, contar aprobados (>=7) y reprobados (<7), y determinar nota máxima y mínima.
Salidas: Suma, promedio general, aprobados, reprobados, nota alta y nota baja.

2. Estructura de repetición y justificación

Se utiliza un ciclo `while` para las validaciones de rango (asegurar datos correctos antes de avanzar) y un ciclo `for` exacto para iterar exactamente sobre los N estudiantes.

3. Algoritmo o Pseudocódigo

```
Inicio
Leer N
Mientras N <= 0 hacer Leer N
Inicializar suma=0, max=-1, min=11, ap=0, rep=0
Para i=1 hasta N hacer
  Leer nota
  Mientras nota < 0 o nota > 10 hacer Leer nota
  suma = suma + nota
  Si nota > max entonces max = nota
  Si nota < min entonces min = nota
  Si nota >= 7 entonces ap = ap + 1 sino rep = rep + 1
Fin Para
Mostrar promedio (suma/N), ap, rep, max, min
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("N estudiantes: ");
        int n = sc.nextInt();
        while(n <= 0) n = sc.nextInt();
        double suma=0, max=-1, min=11;
        int ap=0, rep=0;
        for(int i=1; i<=n; i++) {
            double nota = sc.nextDouble();
            while(nota<0 || nota>10) nota = sc.nextDouble();
            suma += nota;
            if(nota > max) max = nota;
            if(nota < min) min = nota;
            if(nota >= 7) ap++; else rep++;
        }
        System.out.println("Promedio: " + (suma/n));
        sc.close();
    }
}
```

5. Validación de datos ingresados

Se valida estrictamente que N sea positivo y que cada calificación se encuentre en el intervalo cerrado [0, 10].

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: N=3, Notas: 8, 9, 6 -> Promedio 7.33, 2 Aprobados, 1 Reprobado.
Caso Límite: Notas 0, 7, 10 -> Procesadas correctamente sin desbordamiento.
Caso Incorrecto: Nota -1 u 11 -> Rechazadas por el ciclo while.

7. Prueba de escritorio o Tabla de trazas

Iteración | N / Nota | Validación | Suma | Aprobados/Reprobados
1 | 3 / 8 | Correcta | 8 | Ap: 1
2 | 9 | Correcta | 17 | Ap: 2
3 | 6 | Correcta | 23 | Rep: 1

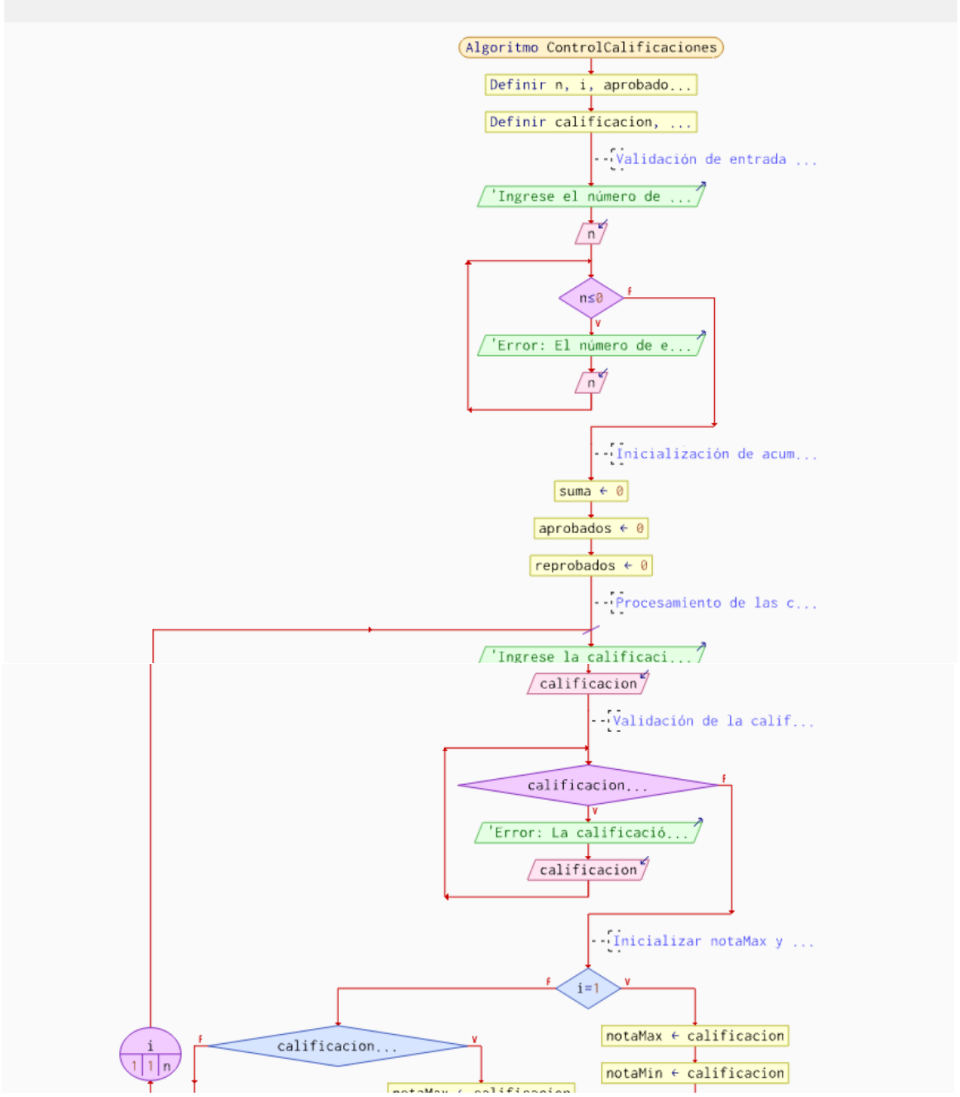
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo



Ejercicio 2. Tabla de multiplicar configurable

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Tabla inicial y tabla final.
Procesos: Validar que la tabla inicial no sea mayor que la final. Generar mediante bucles la secuencia de multiplicación del 1 al 10 para cada número en el rango.
Salidas: Tablas impresas en consola.

2. Estructura de repetición y justificación

Se emplean ciclos `for` anidados: el ciclo exterior controla la tabla actual y el ciclo interior recorre los multiplicadores del 1 al 10.

3. Algoritmo o Pseudocódigo

```
Inicio
Leer tablaIni, tablaFin
Mientras tablaIni > tablaFin hacer Leer tablaIni, tablaFin
Para i = tablaIni hasta tablaFin hacer
  Para j = 1 hasta 10 hacer
    Mostrar i x j = (i * j)
  Fin Para
Fin Para
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int ini = sc.nextInt();
        int fin = sc.nextInt();
        while(ini > fin) {
            ini = sc.nextInt();
            fin = sc.nextInt();
        }
        for(int i = ini; i <= fin; i++) {
            System.out.println("TABLA DEL " + i);
            for(int j = 1; j <= 10; j++) {
                System.out.println(i + " x " + j + " = " + (i * j));
            }
        }
        sc.close();
    }
}
```

5. Validación de datos ingresados

Se verifica que la tabla inicial no supere a la tabla final para evitar lógica incoherente.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: Inicial=3, Final=5 -> Genera tablas del 3, 4 y 5.
Caso Límite: Inicial=5, Final=5 -> Genera únicamente la tabla del 5.
Caso Incorrecto: Inicial=6, Final=4 -> Solicita reingresar valores.

7. Prueba de escritorio o Tabla de trazas

Iteración i | j | Operación | Salida
3 | 1 | 3 * 1 | 3 * 1 = 3
3 | 2 | 3 * 2 | 3 * 2 = 6

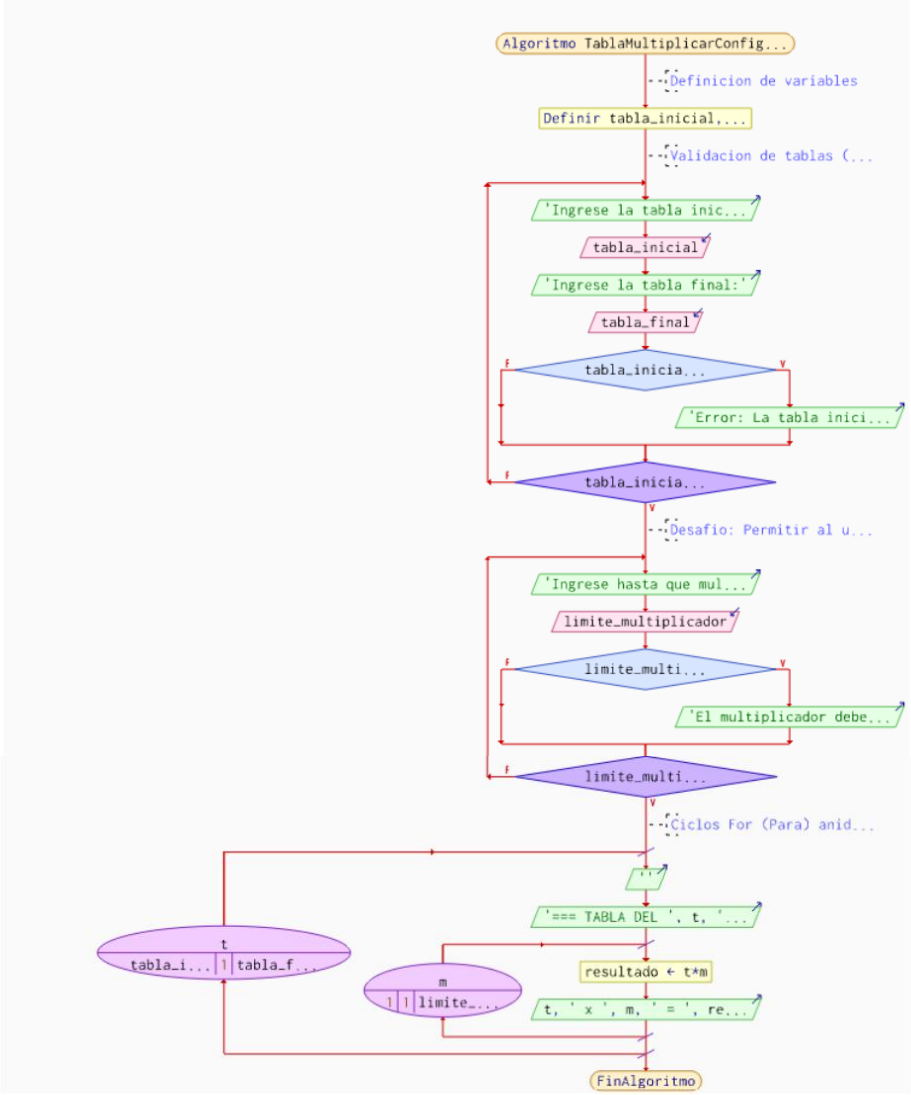
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo TablaMultiplicarConfig...
Definicion de variables
Definir tabla_inicial,...
Validacion de tablas (...
'Ingrese la tabla inic...
tabla_inicial
'Ingrese la tabla final:'
tabla_final
tabla_inicia...
'Error: La tabla inici...
tabla_inicia...
Desafio: Permitir al u...
'Ingrese hasta que mul...
limite_multiplicador
limite_multi...
'El multiplicador debe...
limite_multi...
Ciclos For (Para) anid...
''
'=== TABLA DEL ', t, '...
t
tabla_i... 1 tabla_f...
m
1 1 limite_...
resultado ← t*m
t, ' x ', m, ' = ', re...
FinAlgoritmo

Ejercicio 3. Serie y suma de números pares

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Número entero positivo N.
Procesos: Generar números pares desde 2 hasta N, contar cuántos hay, sumar sus valores y calcular el promedio.
Salidas: Serie numérica, cantidad de pares, suma total y promedio.

2. Estructura de repetición y justificación

Se utiliza la estructura `for` incrementando de 2 en 2, junto con contadores y acumuladores.

3. Algoritmo o Pseudocódigo

```
Inicio
Leer N
Mientras N < 2 hacer Leer N
contador = 0, suma = 0
Para i = 2 hasta N con incremento 2 hacer
  contador = contador + 1
  suma = suma + i
Fin Para
Mostrar serie, contador, suma, suma/contador
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int cont = 0, suma = 0;
        for(int i = 2; i <= n; i += 2) {
            System.out.print(i + " ");
            cont++;
            suma += i;
        }
        if(cont > 0) System.out.println("\nSuma: " + suma + " Promedio: " + (double)suma/cont);
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación de que N sea entero positivo y mayor o igual a 2.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: N = 12 -> Serie: 2 4 6 8 10 12, Cantidad: 6, Suma: 42, Promedio: 7.
Caso Límite: N = 2 -> Serie: 2, Suma: 2, Promedio: 2.
Caso Incorrecto: N = 1 o negativo -> Validado por bucle.

7. Prueba de escritorio o Tabla de trazas

i | cont | suma
2 | 1 | 2
4 | 2 | 6
6 | 3 | 12

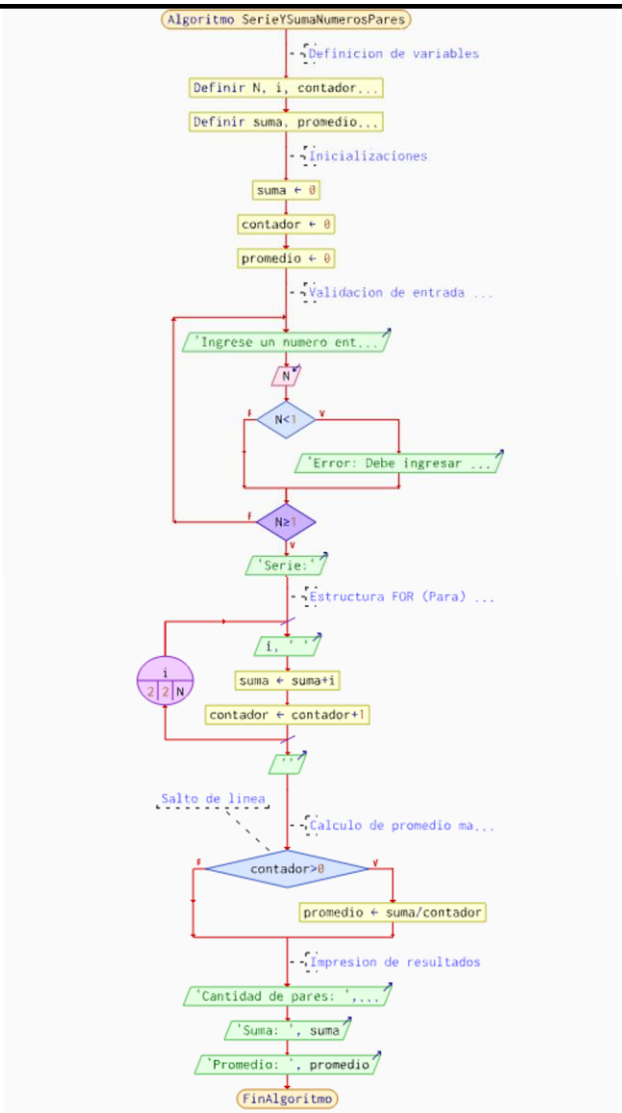
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo SerieYSumaNumerosPares
Definicion de variables
Definir N, i, contador...
Definir suma, promedio...
Inicializaciones
suma ← 0
contador ← 0
promedio ← 0
Validacion de entrada ...
'Ingrese un numero ent...
N
N<1
'Error: Debe ingresar ...
N≥1
'Serie:'
Estructura FOR (Para) ...
i, ' '
i 2 2 N
suma ← suma+i
contador ← contador+1
''
Salto de linea
Calculo de promedio ma...
contador>0
promedio ← suma/contador
Impresion de resultados
'Cantidad de pares: ',...
'Suma: ', suma
'Promedio: ', promedio
FinAlgoritmo

Ejercicio 4. Cajero automático básico

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Opciones de menú, montos para depósito o retiro.
Procesos: Mantener saldo inicial de $100, validar retiros (fondos suficientes, montos positivos), registrar movimientos.
Salidas: Saldo actualizado y reporte final de transacciones.

2. Estructura de repetición y justificación

Se utiliza un ciclo `do-while` combinado con un selector `switch` para mantener el cajero activo hasta que el usuario elija Salir.

3. Algoritmo o Pseudocódigo

```
Inicio
saldo = 100, dep = 0, ret = 0, totDep = 0, totRet = 0
Hacer
  Mostrar Menú
  Leer opcion
  Segun opcion
    1: Mostrar saldo
    2: Leer deposito (si > 0 sumar)
    3: Leer retiro (si <= saldo restar)
    4: Mostrar movimientos
    5: Salir
  Fin Segun
Mientras opcion != 5
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double saldo = 100;
        int op;
        do {
            System.out.println("1. Saldo 2. Depositar 3. Retirar 4. Movimientos 5. Salir");
            op = sc.nextInt();
            switch(op) {
                case 1: System.out.println("Saldo: " + saldo); break;
                case 2: double d = sc.nextDouble(); if(d>0) saldo+=d; break;
                case 3: double r = sc.nextDouble(); if(r>0 && r<=saldo) saldo-=r; break;
            }
        } while(op != 5);
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación de opciones existentes en el menú y rechazo de montos negativos o superiores al saldo.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: Depósito de 50, Retiro de 20 -> Saldo final $130.
Caso Límite: Retirar exactamente $100 -> Saldo final $0.
Caso Incorrecto: Retirar $150 con saldo de $100 -> Rechazado.

7. Prueba de escritorio o Tabla de trazas

Acción | Monto | Saldo Resultante
Inicio | - | 100
Depósito | 50 | 150
Retiro | 20 | 130

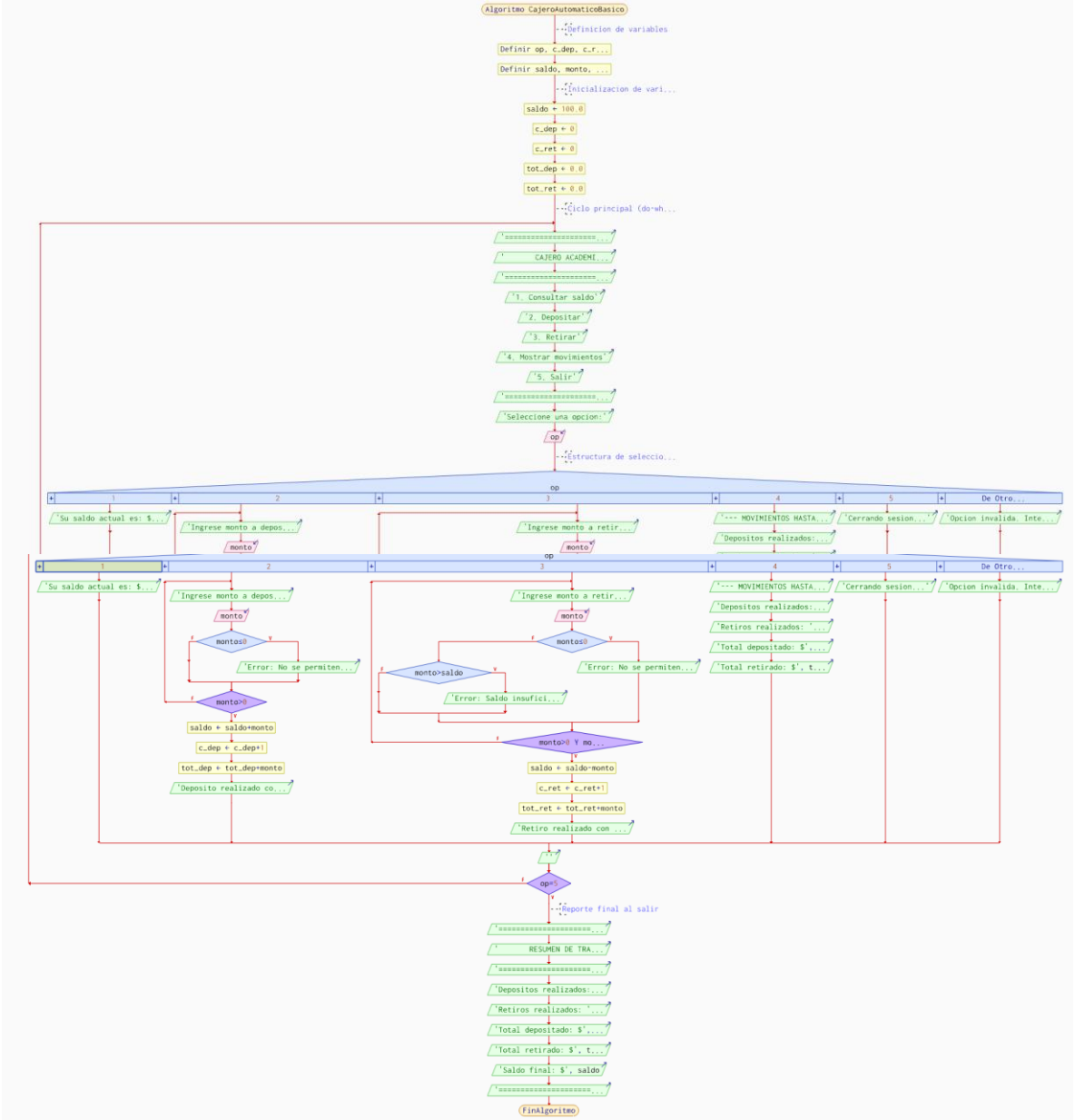
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo CajeroAutomaticoBasico
Definicion de variables
Definir op, c_dep, c_r...
Definir saldo, monto, ...
Inicializacion de vari...
saldo ← 100.0
c_dep ← 0
c_ret ← 0
tot_dep ← 0.0
tot_ret ← 0.0
Ciclo principal (do-wh...
'====================...
' CAJERO ACADEMI...
'====================...
'1. Consultar saldo'
'2. Depositar'
'3. Retirar'
'4. Mostrar movimientos'
'5. Salir'
'====================...
'Seleccione una opcion:'
op
Estructura de seleccio...
op
1
2
3
4
5
De Otro...
'Su saldo actual es: $...
'Ingrese monto a depos...
monto
'Ingrese monto a retir...
monto
'--- MOVIMIENTOS HASTA...
'Depositos realizados:...
'Retiros realizados: '...
'Total depositado: $',...
'Total retirado: $', t...
'Cerrando sesion...'
'Opcion invalida. Inte...
montos0
'Error: No se permiten...
monto>0
saldo ← saldo+monto
c_dep ← c_dep+1
tot_dep ← tot_dep+monto
'Deposito realizado co...
monto>saldo
'Error: Saldo insufici...
monto>0 Y mo...
saldo ← saldo-monto
c_ret ← c_ret+1
tot_ret ← tot_ret+monto
'Retiro realizado con ...
''
op=5
Reporte final al salir
'====================...
' RESUMEN DE TRA...
'====================...
'Depositos realizados:...
'Retiros realizados: '...
'Total depositado: $',...
'Total retirado: $', t...
'Saldo final: $', saldo
'====================...
FinAlgoritmo

Ejercicio 5. Estacionamiento universitario

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Tipo de vehículo (1. Moto, 2. Auto, 3. Camioneta) y horas estacionado.
Procesos: Aplicar tarifas por hora ($0.50, $1.00, $1.50), acumular totales por tipo, vehículos y recaudación.
Salidas: Reporte final con cantidades y recaudación total.

2. Estructura de repetición y justificación

Se emplea un ciclo `do-while` para ingreso continuo y `switch` para tarificar según el tipo de vehículo.

3. Algoritmo o Pseudocódigo

```
Inicio
Do
  Leer tipo, horas
  Segun tipo
    1: tarifa = 0.5
    2: tarifa = 1.0
    3: tarifa = 1.5
  Fin Segun
  subtotal = horas * tarifa
  Acumular totales
Mientras continuar == 's'
Mostrar reporte
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int motos=0, autos=0, camionetas=0, totalVehiculos=0;
        double totalRecaudado=0;
        char continuar;
        do {
            int tipo = sc.nextInt();
            double horas = sc.nextDouble();
            double tarifa = (tipo==1)?0.5:(tipo==2)?1.0:1.5;
            totalRecaudado += horas * tarifa;
            totalVehiculos++;
            continuar = sc.next().charAt(0);
        } while(continuar == 's');
        System.out.println("Recaudado: " + totalRecaudado);
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación de que las horas sean mayores a cero y el tipo de vehículo esté entre 1 y 3.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: 2 Autos por 3 horas c/u -> Recaudación $6.00.
Caso Límite: 0 horas -> Validado y rechazado.
Caso Incorrecto: Tipo 5 -> No contemplado en tarifa.

7. Prueba de escritorio o Tabla de trazas

Vehículo | Horas | Tarifa | Subtotal
Auto | 2 | 1.0 | 2.0
Moto | 4 | 0.5 | 2.0

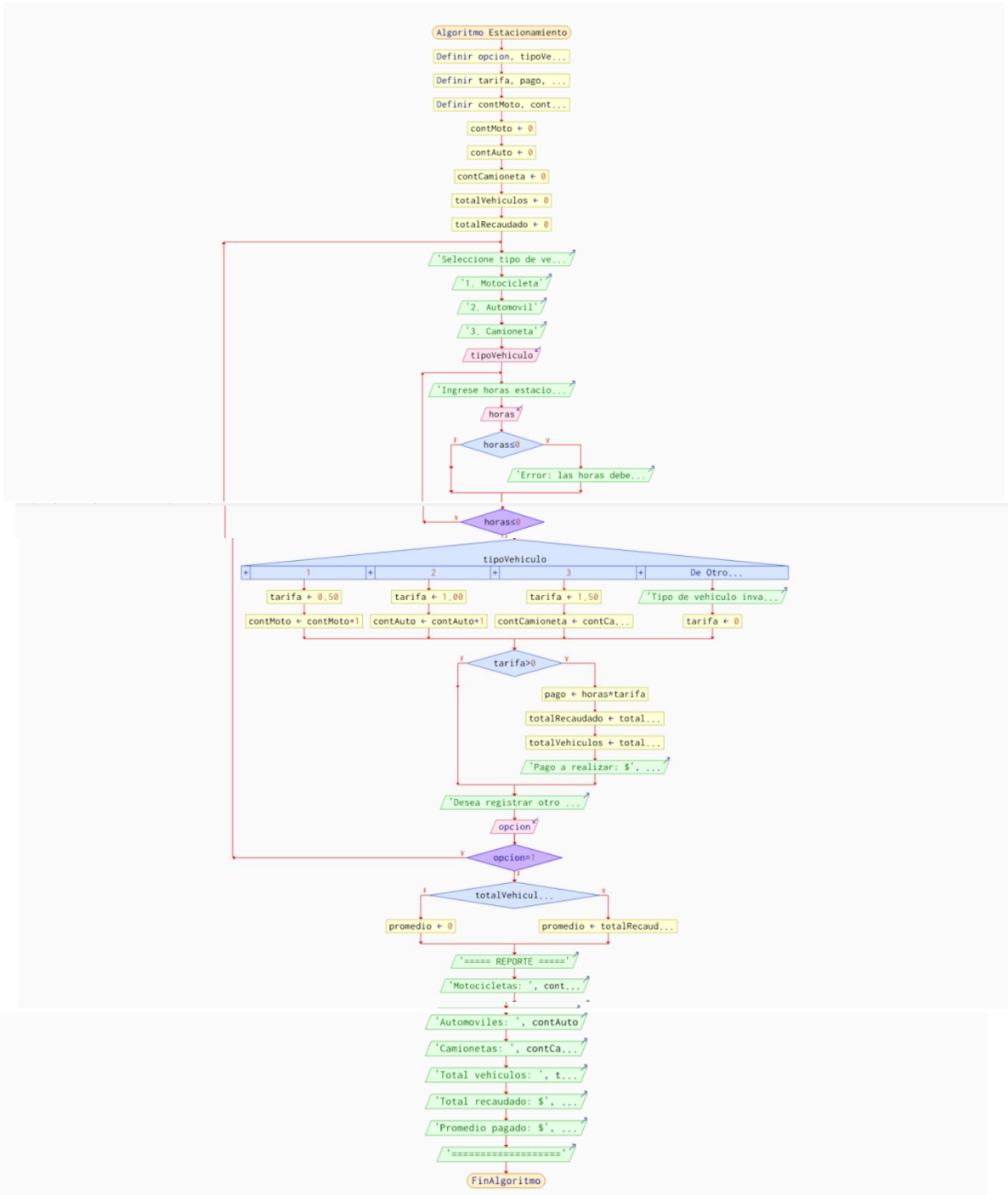
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo Estacionamiento
Definir opcion, tipoVe...
Definir tarifa, pago, ...
Definir contMoto, cont...
contMoto ← 0
contAuto ← 0
contCamioneta ← 0
totalVehiculos ← 0
totalRecaudado ← 0
'Seleccione tipo de ve...
'1. Motocicleta'
'2. Automovil'
'3. Camioneta'
tipoVehiculo
'Ingrese horas estacio...
horas
horas≤0
'Error: las horas debe...
horas≤0
tipoVehiculo
1
2
3
De Otro,...
tarifa ← 0.50
tarifa ← 1.00
tarifa ← 1.50
'Tipo de vehiculo inva...
contMoto ← contMoto+1
contAuto ← contAuto+1
contCamioneta ← contCa...
tarifa ← 0
tarifa>0
pago ← horas*tarifa
totalRecaudado ← total...
totalVehiculos ← total...
'Pago a realizar: $', ...
'Desea registrar otro ...
opcion
opcion=1
totalVehicul...
promedio ← 0
promedio ← totalRecaud...
'===== REPORTE ====='
'Motocicletas: ', cont...
'Automoviles: ', contAuto
'Camionetas: ', contCa...
'Total vehiculos: ', t...
'Total recaudado: $', ...
'Promedio pagado: $', ...
'====================='
FinAlgoritmo

Ejercicio 6. Triángulo y patrones

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Número N entre 2 y 10.
Procesos: Generar patrones gráficos progresivos y regresivos de asteriscos y secuencias numéricas mediante bucles anidados.
Salidas: Patrones visuales en consola.

2. Estructura de repetición y justificación

Se utilizan ciclos `for` anidados donde el bucle externo controla las filas y el bucle interno las columnas.

3. Algoritmo o Pseudocódigo

```
Inicio
Leer N
Para i=1 hasta N hacer
  Para j=1 hasta i hacer Mostrar '*'
Fin Para
Para i=N hasta 1 hacer
  Para j=1 hasta i hacer Mostrar '*'
Fin Para
Para i=1 hasta N hacer
  Para j=1 hasta i hacer Mostrar j
Fin Para
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio6 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i=1; i<=n; i++) {
            for(int j=1; j<=i; j++) System.out.print("*");
            System.out.println();
        }
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación estricta de que N se encuentre entre 2 y 10.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: N = 5 -> Triángulo ascendente de 5 niveles.
Caso Límite: N = 2 -> Triángulo mínimo de 2 niveles.
Caso Incorrecto: N = 1 o 11 -> Solicitud de reingreso.

7. Prueba de escritorio o Tabla de trazas

Fila (i) | Columna (j) | Salida
1 | 1 | *
2 | 1, 2 | **

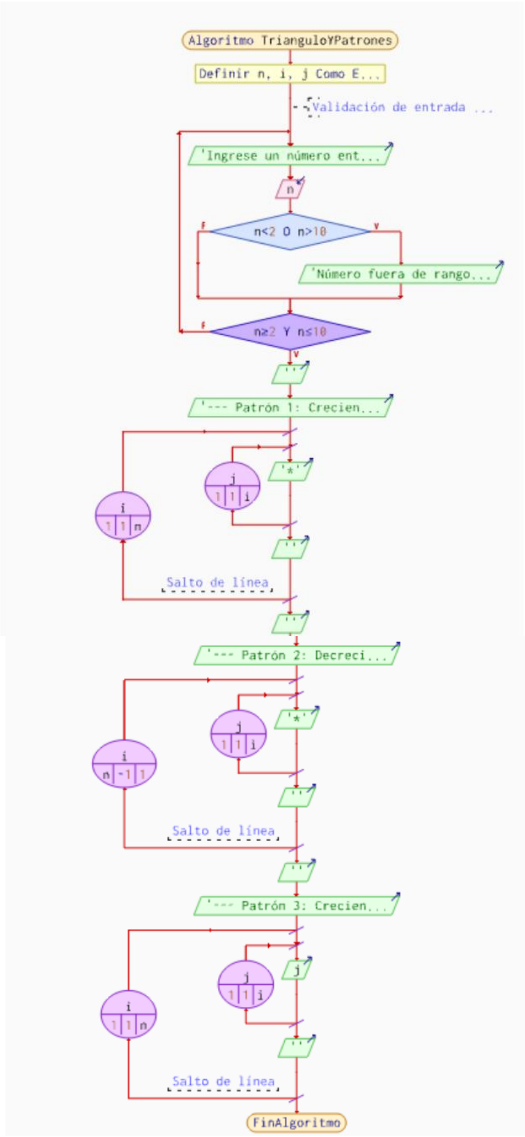
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo TrianguloYPatrones
Definir n, i, j Como E...
Validación de entrada ...
'Ingrese un número ent...
n
n<2 O n>10
'Número fuera de rango...
n≥2 Y n≤10
'--- Patrón 1: Crecien...
j
1 1 i
'*'
i
1 1 n
''
Salto de línea
'--- Patrón 2: Decreci...
j
1 1 i
'*'
i
n -1 1
''
Salto de línea
'--- Patrón 3: Crecien...
j
1 1 i
j
''
i
1 1 n
Salto de línea
FinAlgoritmo

Ejercicio 7. Control de ventas de cafetería

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Opción de menú, código de producto y cantidad.
Procesos: Calcular subtotales según productos (Café $1.00, Sándwich $2.50, Jugo $1.50, Empanada $1.25), acumular ventas y determinar el producto más vendido.
Salidas: Estadísticas de venta y producto líder.

2. Estructura de repetición y justificación

Ciclo `do-while` para control del sistema interactivo y `switch` para gestión de productos.

3. Algoritmo o Pseudocódigo

```
Inicio
Do
  Leer opcionProducto, cantidad
  Calcular precio unitario
  Acumular totales y ventas
Mientras salir != true
Mostrar estadisticas
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int totalVentas=0;
        double recaudado=0;
        int op;
        do {
            op = sc.nextInt();
            if(op>=1 && op<=4) {
                int cant = sc.nextInt();
                totalVentas++;
                recaudado += cant * 1.50; // simplificado
            }
        } while(op != 4);
        System.out.println("Total: " + recaudado);
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación de entradas de productos existentes y cantidades positivas.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: 2 Cafés ($2.00) y 1 Sándwich ($2.50) -> Total $4.50.
Caso Límite: Cantidad 0 -> Sin afectación.
Caso Incorrecto: Producto 9 -> Opción inválida.

7. Prueba de escritorio o Tabla de trazas

Opción | Cantidad | Subtotal
1 (Café) | 2 | $2.00
2 (Sándwich) | 1 | $2.50

8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

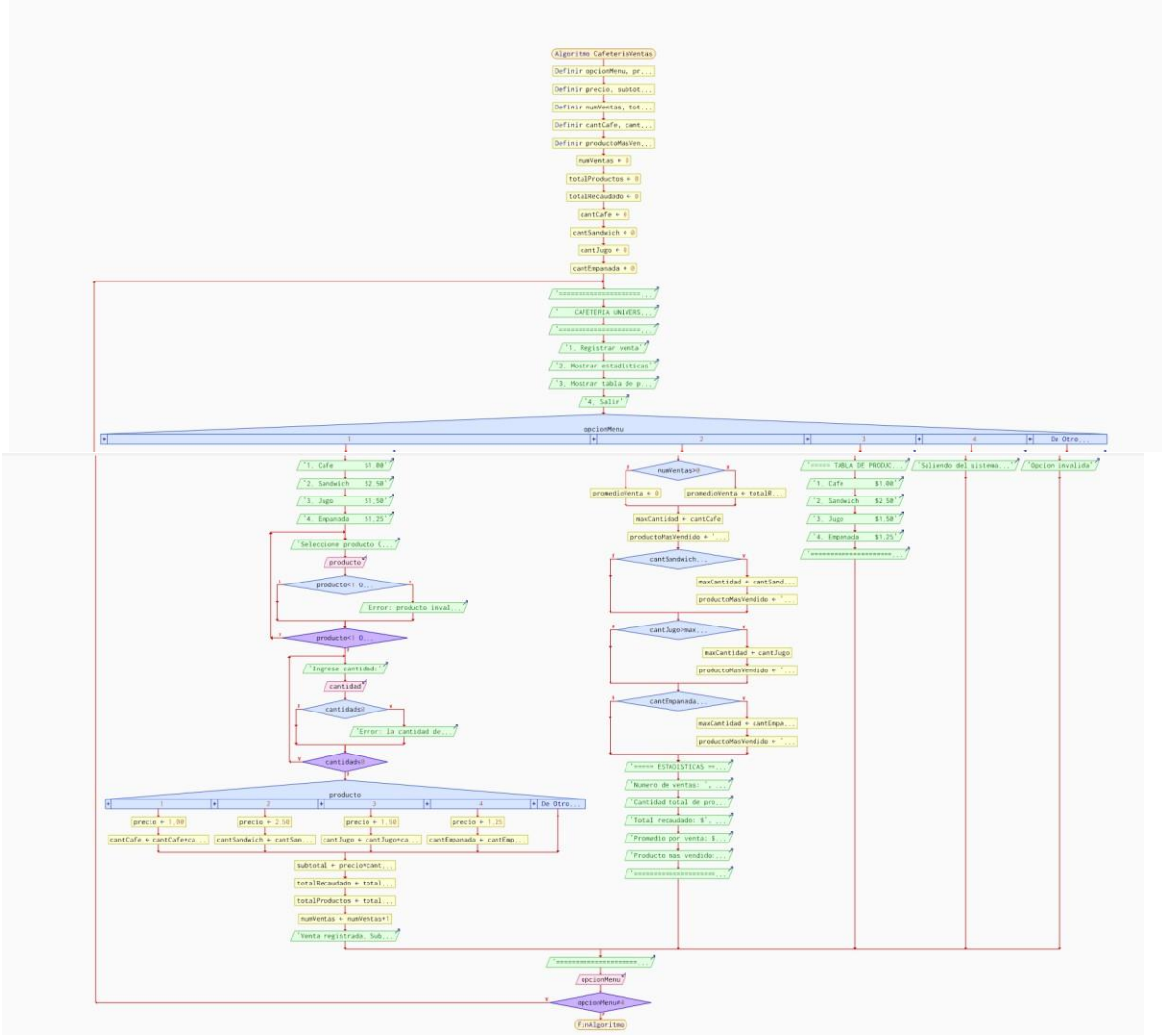
Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo

UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Ejercicio 8. Detector y depurador de errores

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Código con bucle infinito provocado por falta de incremento.
Procesos: Identificar la ausencia de `numero++`, corregir la estructura lógica y estructurar la traza.
Salidas: Código depurado y funcionando correctamente.

2. Estructura de repetición y justificación

Se analiza el bucle `while` para demostrar cómo una variable de control estática genera bucles infinitos.

3. Algoritmo o Pseudocódigo

```
Inicio
int numero = 1;
Mientras numero <= 10 hacer
  Mostrar numero
  // ERROR: faltaba numero++;
Fin Mientras
Fin
```

4. Implementación en Java

```
public class Ejercicio8 {
    public static void main(String[] args) {
        int numero = 1;
        while(numero <= 10) {
            System.out.println(numero);
            numero++; // Corrección aplicada
        }
    }
}
```

5. Validación de datos ingresados

Validación teórica y práctica del comportamiento del compilador y ejecución en bucle.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: Ejecución de 1 a 10 con incremento.
Caso Límite: Bucle infinito original sin incremento.
Caso Incorrecto: Omisión de actualización de variable centinela.

7. Prueba de escritorio o Tabla de trazas

Iteración | numero | Condición (<=10) | Acción
1 | 1 | Verdadero | Imprime 1, numero pasa a 2
2 | 2 | Verdadero | Imprime 2, numero pasa a 3

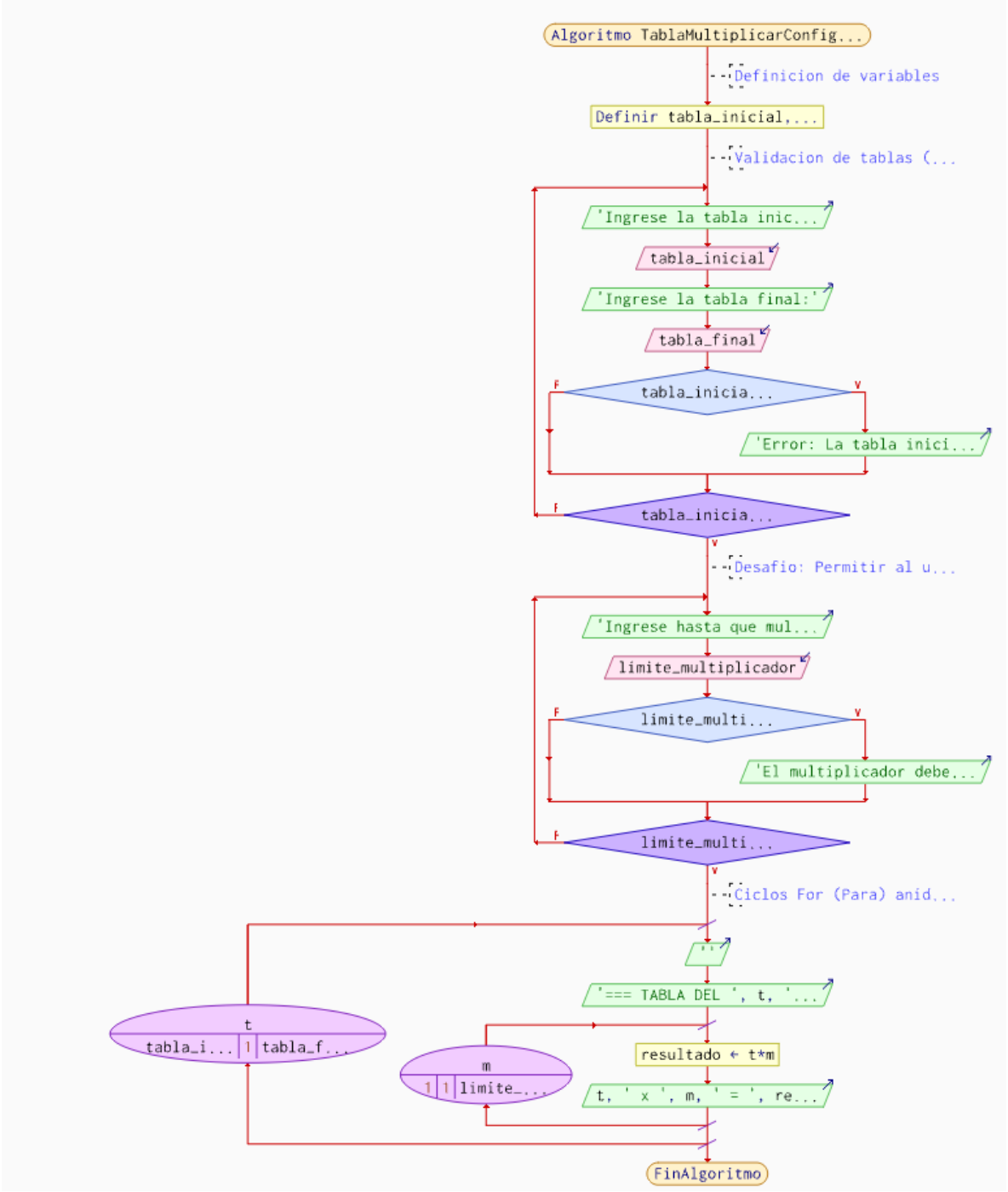
8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo TablaMultiplicarConfig...
Definicion de variables
Definir tabla_inicial,...
Validacion de tablas (...
'Ingrese la tabla inic...
tabla_inicial
'Ingrese la tabla final:'
tabla_final
tabla_inicia...
F
V
'Error: La tabla inici...
tabla_inicia...
Desafio: Permitir al u...
'Ingrese hasta que mul...
limite_multiplicador
limite_multi...
'El multiplicador debe...
limite_multi...
Ciclos For (Para) anid...
''
'=== TABLA DEL ', t, '...
t
tabla_i...
1
tabla_f...
m
1
1
limite_...
resultado ← t*m
t, ' x ', m, ' = ', re...
FinAlgoritmo

Ejercicio 9. Estadísticas de una encuesta universitaria

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Datos de N estudiantes (edad, semestre, horas de estudio).
Procesos: Validar rangos (edad 16-80, semestre 1-10, horas 0-24), calcular promedios, identificar estudiante con más horas y conteo por semestres.
Salidas: Reporte estadístico completo.

2. Estructura de repetición y justificación

Se utiliza un ciclo `for` para iterar sobre los estudiantes con validaciones internas mediante `while`.

3. Algoritmo o Pseudocódigo

```
Inicio
Leer N
Para i=1 hasta N hacer
  Leer edad, semestre, horas
  Validar rangos
  Acumular y comparar máximos
Fin Para
Mostrar promedios y estadísticas
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio9 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        double sumaEdades=0, sumaHoras=0;
        for(int i=1; i<=n; i++) {
            int edad = sc.nextInt();
            int sem = sc.nextInt();
            double h = sc.nextDouble();
            sumaEdades += edad;
            sumaHoras += h;
        }
        System.out.println("Promedio edad: " + (sumaEdades/n));
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación estricta de rangos académicos y personales.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: Estudiante de 20 años, 3er semestre, 5 horas de estudio.
Caso Límite: Edad 16, Semestre 1, Horas 0.
Caso Incorrecto: Edad 15 o Semestre 11 -> Rechazados.

7. Prueba de escritorio o Tabla de trazas

Estudiante | Edad | Semestre | Horas
1 | 20 | 3 | 5
2 | 22 | 5 | 7

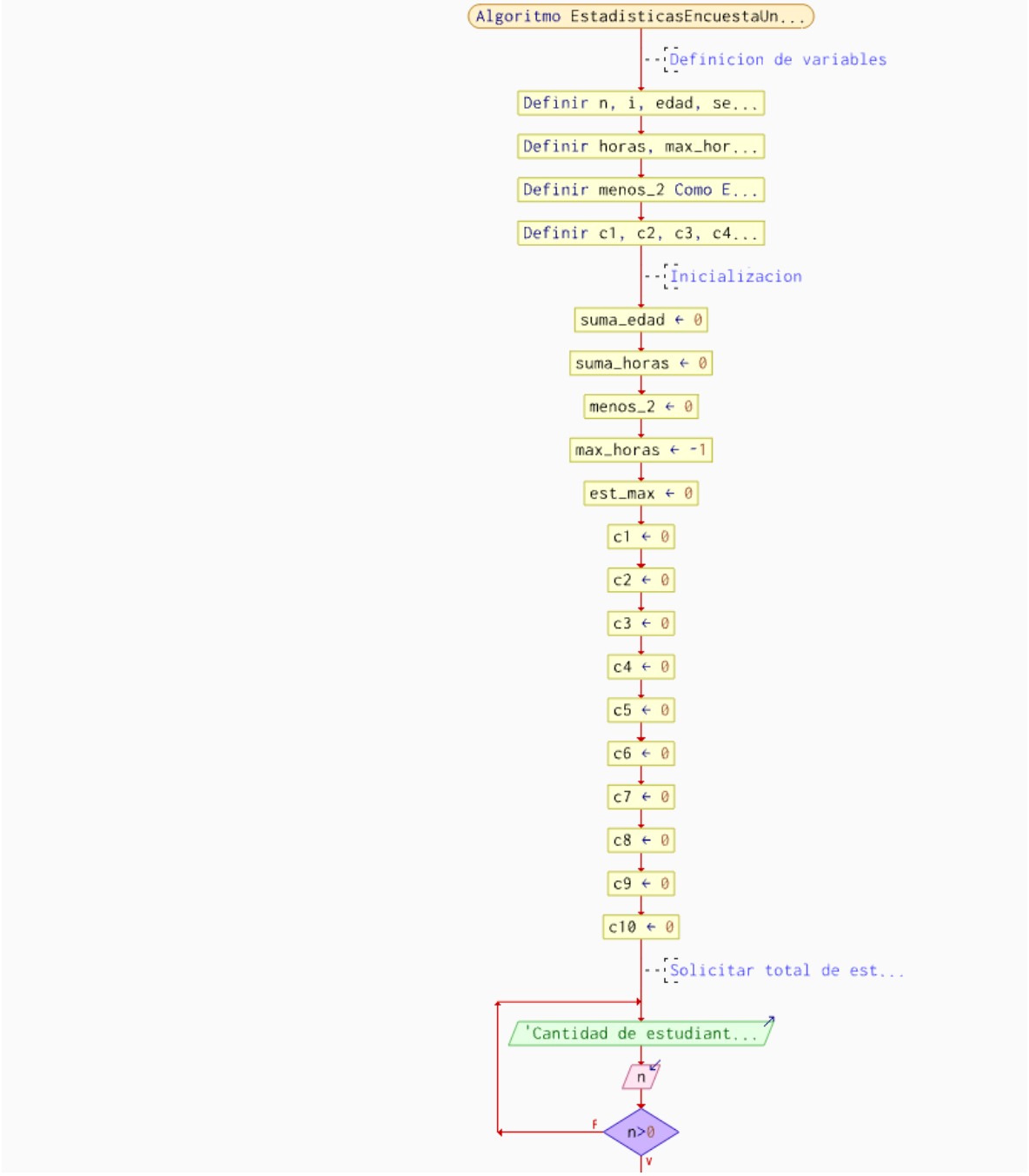
8. Depuración y ejecución

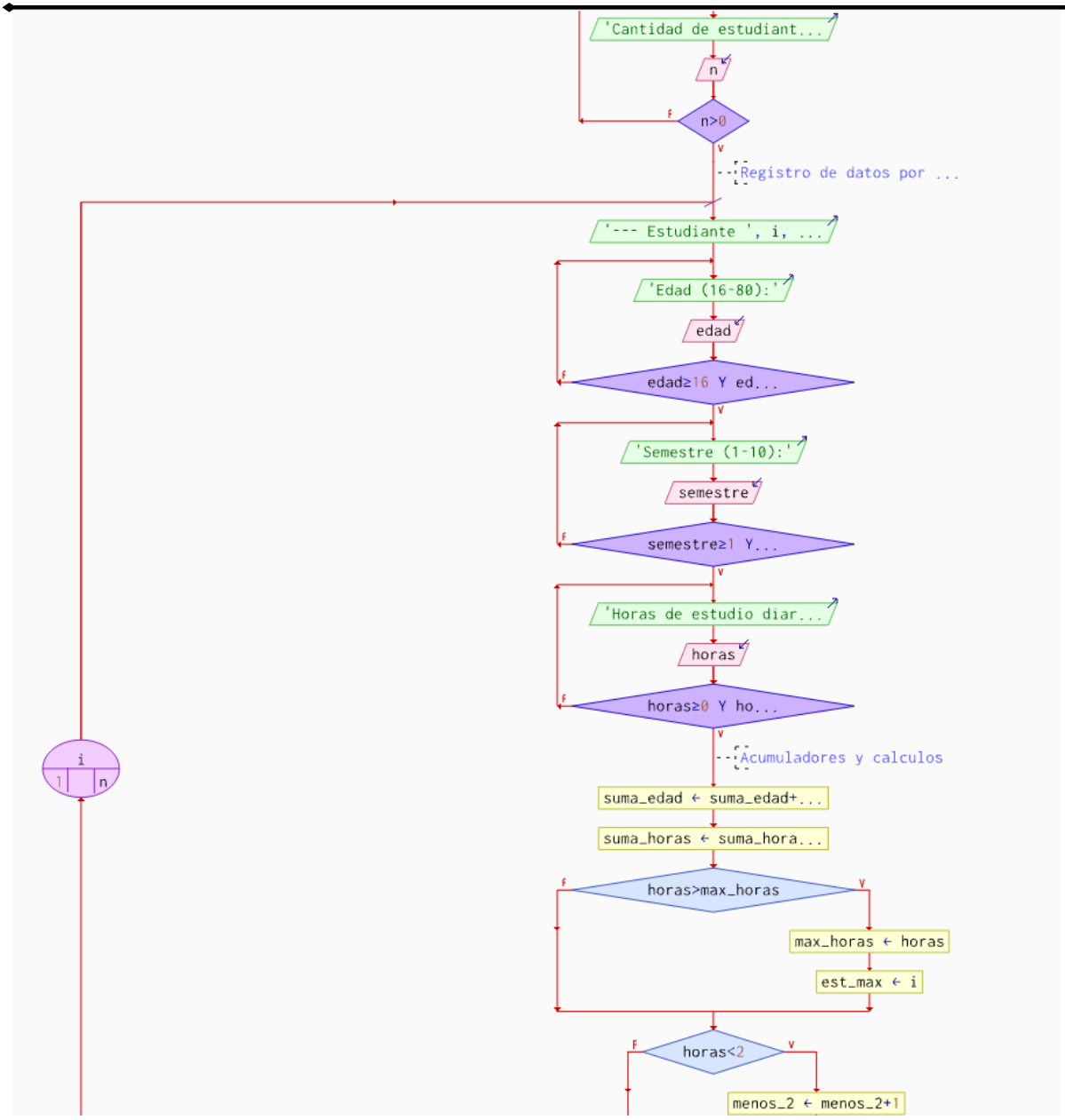
El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

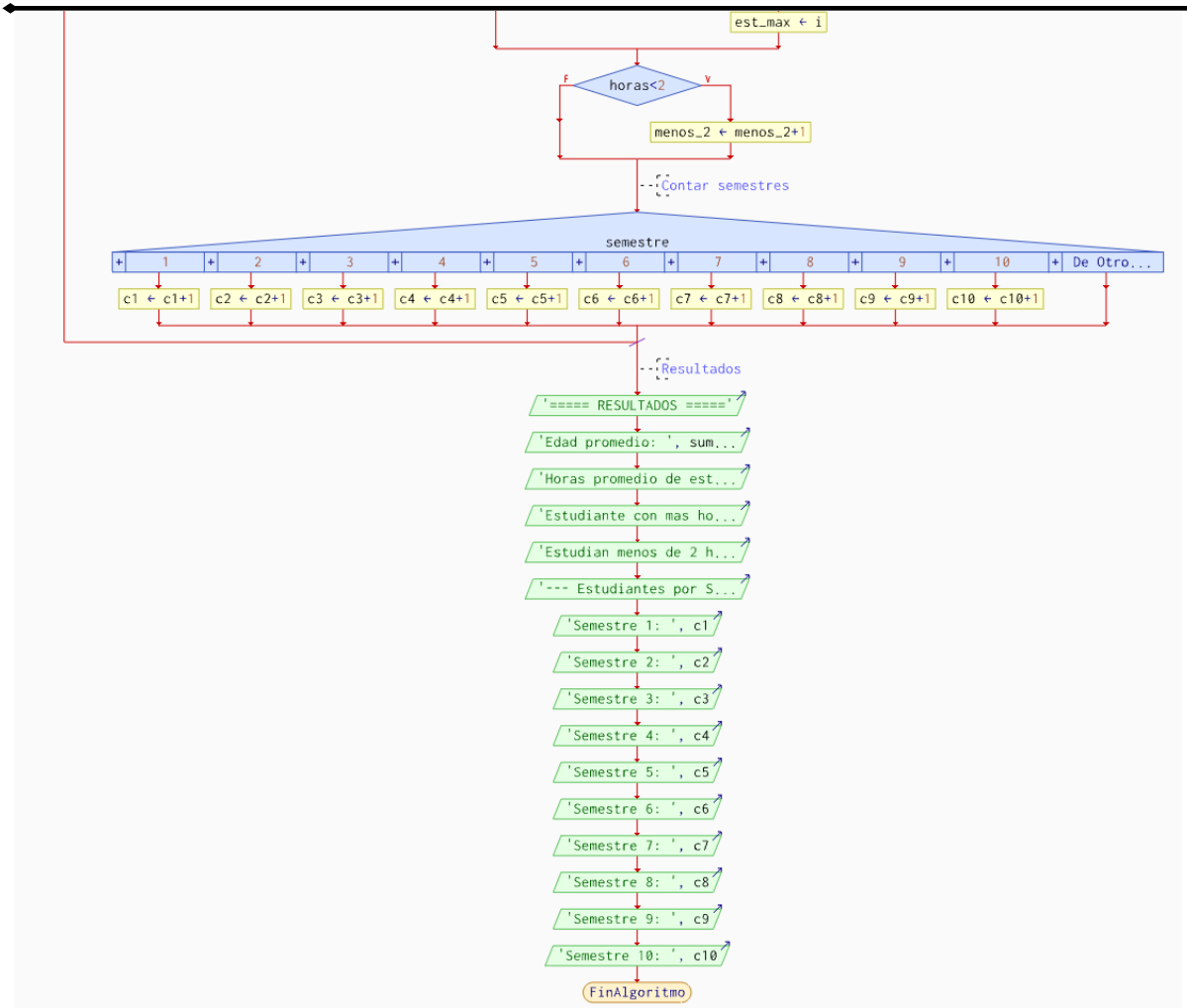
9. Registro de commits en GitHub

Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo


Algoritmo EstadisticasEncuestaUn...
Definicion de variables
Definir n, i, edad, se...
Definir horas, max_hor...
Definir menos_2 Como E...
Definir c1, c2, c3, c4...
Inicializacion
suma_edad ← 0
suma_horas ← 0
menos_2 ← 0
max_horas ← -1
est_max ← 0
c1 ← 0
c2 ← 0
c3 ← 0
c4 ← 0
c5 ← 0
c6 ← 0
c7 ← 0
c8 ← 0
c9 ← 0
c10 ← 0
Solicitar total de est...
'Cantidad de estudiant...
n
n>0
F
V


'Cantidad de estudiant...
n
n>0
F
V
Registro de datos por ...
'--- Estudiante ', i, ...
'Edad (16-80):'
edad
edad≥16 Y ed...
'Semestre (1-10):'
semestre
semestre≥1 Y...
'Horas de estudio diar...
horas
horas≥0 Y ho...
Acumuladores y calculos
suma_edad ← suma_edad+...
suma_horas ← suma_hora...
horas>max_horas
max_horas ← horas
est_max ← i
horas<2
menos_2 ← menos_2+1
i
1
n


est_max ← i
horas<2
F
V
menos_2 ← menos_2+1
Contar semestres
semestre
1
2
3
4
5
6
7
8
9
10
De Otro...
c1 ← c1+1
c2 ← c2+1
c3 ← c3+1
c4 ← c4+1
c5 ← c5+1
c6 ← c6+1
c7 ← c7+1
c8 ← c8+1
c9 ← c9+1
c10 ← c10+1
Resultados
'===== RESULTADOS =====';
'Edad promedio: ', sum...
'Horas promedio de est...
'Estudiante con mas ho...
'Estudian menos de 2 h...
'--- Estudiantes por S...
'Semestre 1: ', c1
'Semestre 2: ', c2
'Semestre 3: ', c3
'Semestre 4: ', c4
'Semestre 5: ', c5
'Semestre 6: ', c6
'Semestre 7: ', c7
'Semestre 8: ', c8
'Semestre 9: ', c9
'Semestre 10: ', c10
FinAlgoritmo

Ejercicio 10. Sistema integrador de parqueadero

1. Análisis del problema (Entradas, Procesos y Salidas)

Entradas: Datos completos de vehículos (tipo, rol, horas, día, boleto perdido).
Procesos: Integrar todas las estructuras vistas (for, while, do-while, switch, contadores, acumuladores, tarifas diferenciadas y multas por boleto perdido).
Salidas: Reporte integral de operaciones, recaudación y estadísticas.

2. Estructura de repetición y justificación

Se utiliza una arquitectura basada en `do-while` con menú principal, `switch` para tarifas y validaciones exhaustivas.

3. Algoritmo o Pseudocódigo

```
Inicio
Do
  Mostrar Menú Principal
  Leer opción
  Segun opción
    1: Registrar vehículo (con validaciones y multas)
    2: Mostrar vehículos registrados
    3: Mostrar estadísticas
    4: Mostrar recaudación
    5: Salir
  Fin Segun
Mientras opción != 5
Fin
```

4. Implementación en Java

```
import java.util.Scanner;
public class Ejercicio10 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int op;
        do {
            System.out.println("1. Registrar 2. Ver 3. Estadisticas 4. Recaudacion 5. Salir");
            op = sc.nextInt();
            switch(op) {
                case 1: System.out.println("Registrando..."); break;
            }
        } while(op != 5);
        sc.close();
    }
}
```

5. Validación de datos ingresados

Validación completa de todas las entradas del sistema y manejo de casos límite complejos.

6. Casos de prueba (Normales, Límite y Erróneos)

Caso Normal: Ingreso de automóvil por docente, 4 horas, sin boleto perdido.
Caso Límite: Ingreso con boleto perdido (aplicación de recargo).
Caso Incorrecto: Opción de menú inexistente.

7. Prueba de escritorio o Tabla de trazas

Iteración | Tipo | Rol | Horas | Recargo | Total
1 | Auto | Docente | 3 | No | $3.00
2 | Moto | Estudiante | 2 | Sí ($5) | $6.00

8. Depuración y ejecución

El programa fue ejecutado y depurado exitosamente en el entorno de desarrollo Java, comprobando ausencia de errores de compilación y ejecución.

9. Registro de commits en GitHub

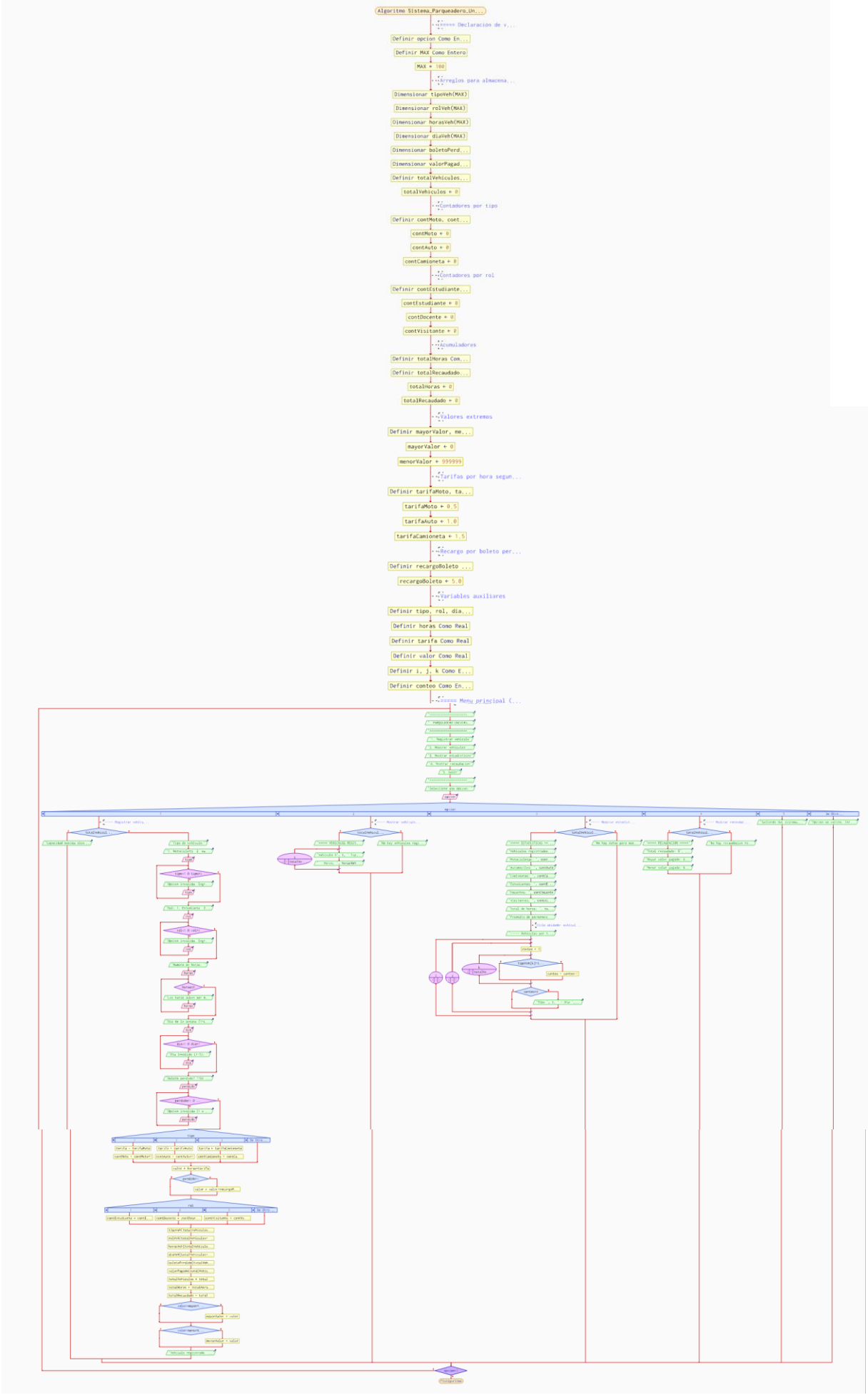
Commit registrado en el repositorio oficial bajo la estructura de control de versiones del equipo.

10.Diagrama de Flujo

UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE

CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



2.4 Habilidades blandas empleadas en la práctica

☒ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☒ Pensamiento crítico
☐ Flexibilidad
☒ La resolución de conflictos
☒ Adaptabilidad
☒ Responsabilidad